



UNIVERSITÀ
DI PISA

Università di Pisa
Dipartimento di Informatica
Corso di Laurea Magistrale in Informatica

Business Process Modeling

Relazione Progetto P53: Pesca

Analisi dei modelli BPMN e delle reti di workflow

Fabio Piscitelli — Matricola 715339

Anno Accademico 2025/2026

1 Introduzione

L'obiettivo del progetto è modellare lo scenario in cui due amici, Alex e Bob, si accordano d'estate per andare a pescare in mare. Alex, proprietario della barca, invia a Bob la lista delle proprie disponibilità. Bob può rispondere in tre modi: proporre una data e il tipo di attrezzatura, delegare la scelta ad Alex, oppure comunicare di non essere disponibile, concludendo il processo. Se Bob invia una proposta, Alex la accetta o ne formula una diversa; se Bob delega, Alex invia una propria proposta. In seguito alla risposta di Alex, Bob può confermare, fare una controproposta oppure annullare l'appuntamento. Raggiunto l'accordo, Alex esegue la manutenzione della barca e comunica a Bob il luogo dell'appuntamento; se durante la manutenzione emergono problemi al motore, i due devono riaccordarsi per una data successiva.

Trattandosi di uno scenario con più attori che si scambiano messaggi, si è scelta la notazione BPMN, che permette di descrivere sia l'*orchestration* dei singoli partecipanti sia la loro *collaboration*. I diagrammi BPMN sono stati realizzati con Camunda Modeler; le reti di workflow sono state ottenute per traduzione dai diagrammi e analizzate con WoPeD. La relazione descrive i modelli prodotti, la traduzione in reti di Petri e le proprietà delle workflow net risultanti.

2 Diagrammi BPMN

Nello scenario si individuano due attori, Alex e Bob, modellati come due *pool* distinte con relativi processi di orchestration; i due processi comunicano tra loro nel diagramma di collaboration tramite *message flow*.

2.1 Alex

Il processo di Alex inizia con l'invio della lista delle disponibilità (*send task* "Invia lista disponibilità"). Alex resta poi in attesa della risposta di Bob con un unico *intermediate catch event* ("Ricevi risposta disponibilità"): in base al contenuto ricevuto (proposta, delega o indisponibilità) uno split XOR indirizza il flusso sul ramo corretto. Se Bob ha proposto, Alex valuta la data e, con uno split XOR ("Proposta accettabile?"), accetta o invia una controproposta; se Bob ha delegato, Alex invia una nuova proposta. Dopo aver inviato una proposta, Alex attende la risposta di Bob (accettazione, controproposta o annullamento). Raggiunto l'accordo, Alex esegue la manutenzione della barca, comunica l'esito e, con lo split XOR "Manutenzione avvenuta con successo?", o comunica il luogo dell'appuntamento e termina, oppure ricomincia la negoziazione inviando nuovamente la lista. Il diagramma è mostrato in Fig. 1.

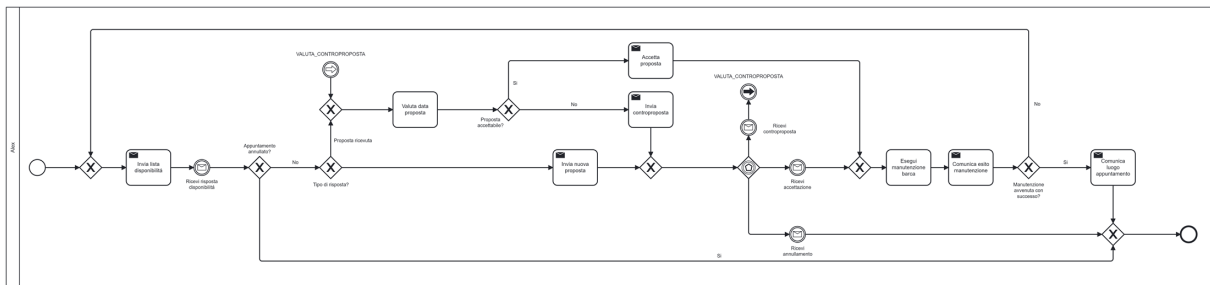


Figura 1: Diagramma BPMN del processo di Alex.

2.2 Bob

Il processo di Bob inizia ricevendo la lista di disponibilità (*receive task* "Ricevi lista disponibilità"). Con uno split XOR ("Come rispondo?") Bob sceglie tra proporre data e attrezzatura, delegare la scelta ad Alex o comunicare l'indisponibilità (che conclude il processo). Se propone

o delega, Bob attende la risposta di Alex; alla ricezione valuta la proposta e, con lo split XOR “Come rispondo?”, decide se accettare, fare una controproposta o annullare l’appuntamento. Raggiunto l’accordo, Bob riceve l’esito della manutenzione e, in base ad esso, o riceve il luogo dell’appuntamento e termina, oppure ricomincia la negoziazione. Il diagramma è mostrato in Fig. 2.

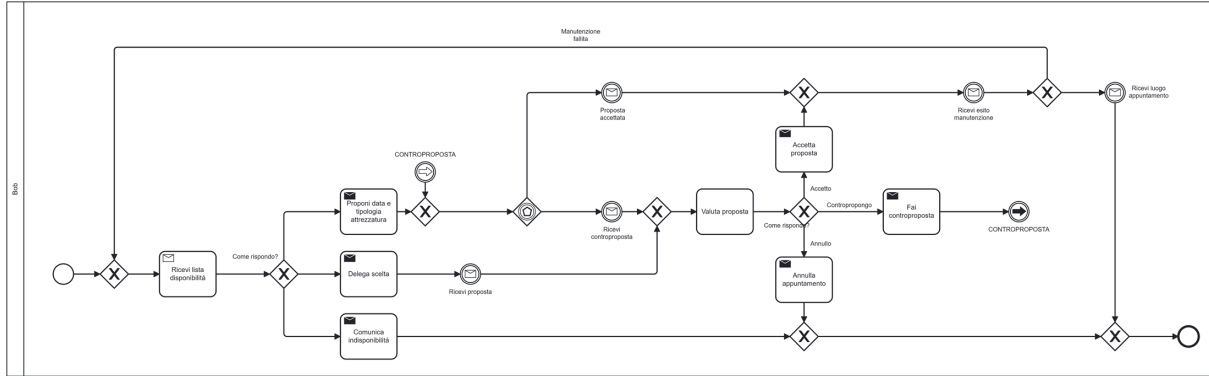


Figura 2: Diagramma BPMN del processo di Bob.

2.3 Collaboration

Il diagramma di collaboration (Fig. 3) unisce i due processi collegandoli con i message flow tra i corrispondenti send task ed eventi/riceive task. Complessivamente sono presenti dodici message flow. Un aspetto peculiare dello scenario è che i tre possibili messaggi iniziali di Bob (proposta, delega, indisponibilità) confluiscono su un *unico* intermediate catch event di Alex: questa scelta di modellazione richiede un trattamento particolare in fase di traduzione, discusso nella Sezione 3.1.

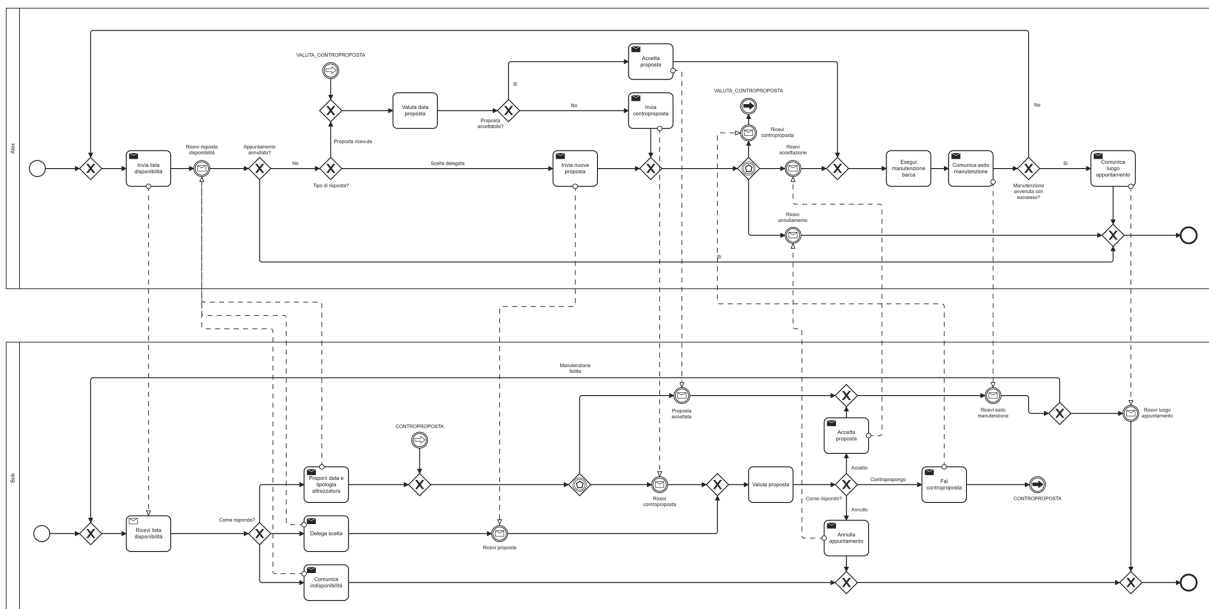


Figura 3: Diagramma BPMN della collaboration tra Alex e Bob.

3 Reti di workflow

I diagrammi BPMN sono stati tradotti in reti di workflow per poterne verificare formalmente le proprietà. Ogni processo di orchestration è stato trasformato in un *workflow module* e analizzato

singolarmente; i due module sono poi stati composti nel *workflow system* corrispondente al diagramma di collaboration.

3.1 Traduzione

I criteri di traduzione adottati sono i seguenti:

- ogni task ed evento è stato trasformato in una transizione, e ogni arco di sequence flow in un posto;
- gli split e i join AND/XOR sono stati resi con operatori annotati con il tipo corretto (lo “zucchero sintattico” di WoPeD), corrispondenti alla opportuna combinazione di posti e transizioni;
- sono stati aggiunti un unico posto iniziale e un unico posto finale, collegati ai punti di start/end dei due module tramite transizioni, in modo da ottenere un’unica workflow net anche per il sistema complessivo;
- ogni message flow è stato tradotto in un *posto di interfaccia* (canale) che collega la transizione mittente a quella ricevente.

Lo scenario presenta due punti che, tradotti in modo ingenuo, comprometterebbero la soundness del sistema e che sono stati risolti aggiungendo posti e transizioni dedicati.

Catch unico per tre messaggi. L’intermediate catch event di Alex riceve uno di tre messaggi alternativi da Bob. Rappresentarlo con una singola transizione di ricezione seguita da uno split XOR “libero” non basta: la scelta del ramo di Alex sarebbe scorrelata dal messaggio effettivamente arrivato, e Alex potrebbe imboccare un ramo diverso da quello atteso da Bob, portando a deadlock. Il catch è stato quindi tradotto con *tre transizioni di ricezione distinte*, una per messaggio, ciascuna guidata dal proprio posto di interfaccia e collegata direttamente al ramo di Alex corrispondente. In questo modo è il messaggio ricevuto a determinare univocamente il percorso, garantendo la corretta interazione.

Esito della manutenzione. Alex comunica a Bob l’esito della manutenzione con un solo messaggio, dopodiché entrambi decidono, con uno split XOR indipendente, se procedere (manutenzione riuscita) o riaccordarsi (manutenzione fallita). Poiché la decisione è presa da Alex ma deve riflettersi anche sul ramo di Bob, le due scelte sono state correlate aggiungendo due posti di interfaccia (“esito OK” ed “esito KO”), prodotti dalla decisione di Alex e consumati dalle transizioni del ramo di Bob. Ciò impedisce a Bob di scegliere un ramo incoerente con l’esito reale.

Complessivamente l’interfaccia del sistema è costituita dai dodici posti dei message flow e dai due posti di correlazione dell’esito.

3.2 Alex

La rete di workflow di Alex è mostrata in Fig. 4–5 (divisa in due parti per leggibilità). Ha 28 posti, 14 transizioni, 9 operatori e 56 archi. È una workflow net: esiste un solo posto iniziale i con $\bullet i = \emptyset$ e un solo posto finale o con $o^\bullet = \emptyset$, e ogni nodo giace su un cammino da i a o . La rete è *safe* (1-bounded) e *free-choice*, ed è *well-structured* (non presenta PT-handle né TP-handle). È inoltre S-coverable, con 51 S-component distinte, dunque ammette un S-invariante positivo; essendo sound, la rete di copertura N^* è bounded e live e ammette un T-invariante positivo. Poiché la rete è safe, il grafo di raggiungibilità è finito e coincide con il grafo di copertura: esso ha 28 vertici e 33 archi. La rete è **sound**.

[illegible]

3.4 Collaboration (workflow system)

- **Non free-choice.** Sono presenti quattro gruppi di violazione del free-choice, corrispondenti esattamente ai punti in cui un posto di interfaccia (messaggio) fa da guardia su una transizione che ha anche altri posti nel pre-set. Sono cioè i “cancelli” dei messaggi scambiati tra Alex e Bob: è proprio la sincronizzazione tramite canali a rendere la rete non free-choice, e non è eliminabile senza snaturare l’interazione.
- **Non well-structured.** La rete presenta 120 PT-handle e 131 TP-handle. È un effetto atteso della composizione di due processi concorrenti sincronizzati da canali: ogni transizione mittente genera due token (di controllo e di messaggio) che seguono cammini disgiunti e riconvergono a valle, producendo un gran numero di handle.

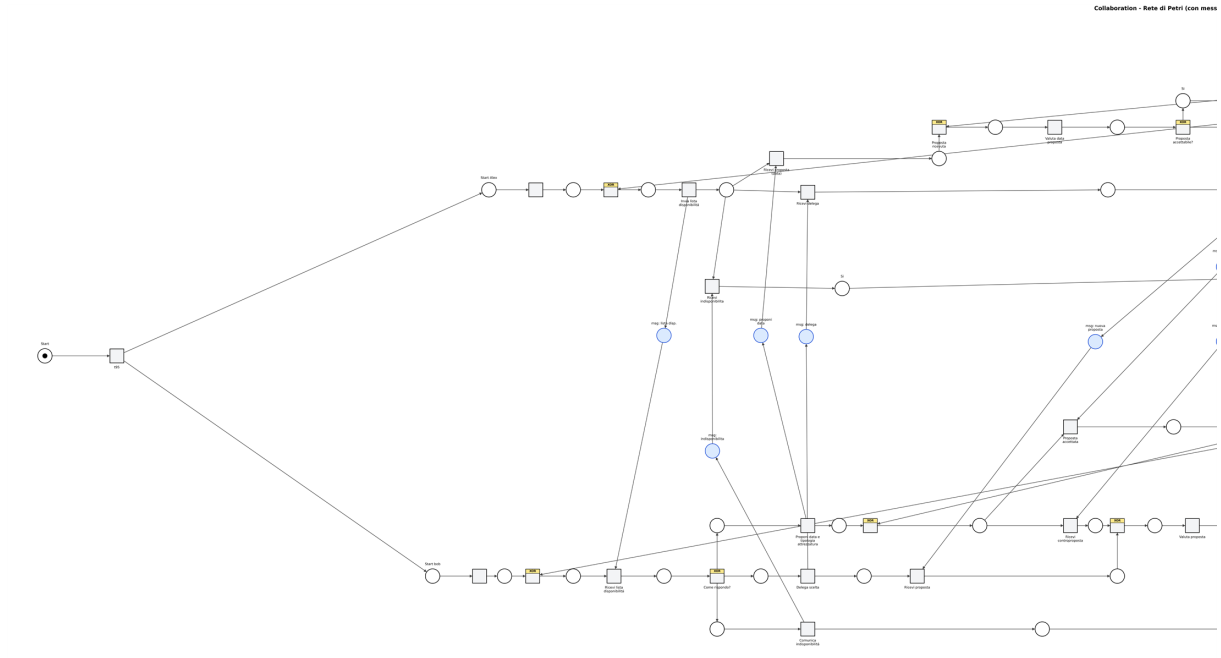


Figura 8: Workflow system: collaboration tra Alex e Bob con i message flow (parte 1 di 2).

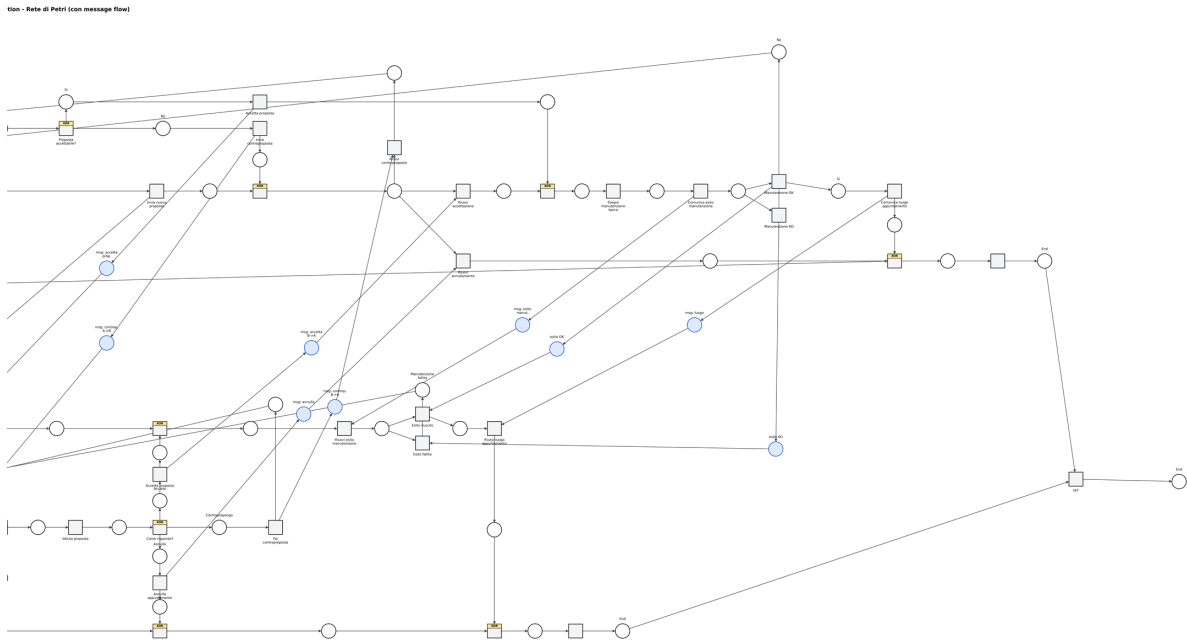


Figura 9: Workflow system: collaboration tra Alex e Bob con i message flow (parte 2 di 2).

Va sottolineato che free-choice e well-structuredness sono condizioni *sufficienti*, non necessarie, per la soundness: la loro assenza non è un errore. La soundness del sistema è stata verificata direttamente sul grafo di raggiungibilità, che ha 199 vertici e 340 archi. Poiché la rete è bounded, il grafo di raggiungibilità è finito e coincide con il grafo di copertura; essendo sound, N^* è bounded e live e la rete è S-coverable, con conseguente esistenza di T-invarianti e S-invarianti positivi.

La Tab. 1 riassume le metriche delle tre reti.

Tabella 1: Metriche strutturali e dimensione del grafo di raggiungibilità (RG).

Rete	Posti	Transizioni	Operatori	Archivi	RG vertici	RG archi
Alex	28	14	9	56	28	33
Bob	30	15	9	59	30	35
Collaboration	72	37	14	149	199	340

4 Conclusioni

Sono stati modellati i due processi di orchestration di Alex e Bob e il loro diagramma di collaboration, quindi tradotti in reti di workflow. Ciascun processo è, isolatamente, una workflow net safe, free-choice, well-structured e sound. Il sistema complessivo, ottenuto componendo i due module tramite i posti di interfaccia dei message flow, è anch'esso **sound**: pur non essendo né free-choice né well-structured, come atteso per una collaboration sincronizzata a messaggi, soddisfa il requisito richiesto dalla consegna per tutti i pool e per l'intero diagramma di collaboration. Il trattamento accurato del catch unico a tre messaggi e la correlazione dell'esito della manutenzione sono stati determinanti per ottenere l'interazione corretta e la soundness del sistema.